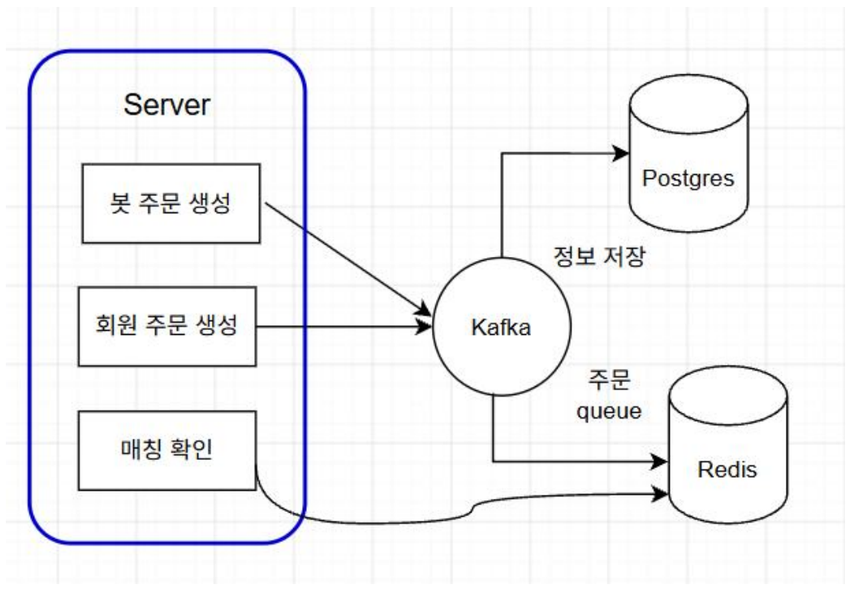
WebSocket 연결 실패율 44% 해결

시스템 리소스 병목 현상을 파악하고 해결한 경험

2026.02 — 2026.04 역할: BE, 팀장 프로젝트: Click-CEO - SSAFY

문제 상황

- Websocket으로 전환 후 화면 멈춤 현상 발생: 호가창을 Polling으로 구현했던 기존 내용을 실시간 성이 떨어진다고 판단되서 Websocket으로 전환
- k6 Test결과 44%의 연결 실패: k6 200 VU Stress Test 의 핸드셰이크 요청이 처리되지 못해 44%의 연결 실패 발생



원인 분석

- 변경되는 WebSocket과 데이터 전송 부분은 사전 개별 테스트를 통해 문제가 없음을 확인
- 봇 주문을 끄고 테스트 결과 k6 테스트 연결 실패율과 CPU 사용량이 낮아짐을 확인
- Redis 데이터를 보는 중 주문 데이터가 시간이 지남에 따라 적재율이 늘어남 및 유실을 확인

AI 한계

- 부하를 고려하지 않은 코드 작성: 병목 구간이 발생한 코드들은 AI로 코드 생성
- 코드 밖에서 생기는 병목은 확인 불가: 데이터가 쌓였을 때에만 network 병목 및 데이터 유실이 발생
- 분석 범위가 Spring 내부코드에서만 머물러 다른 시스템의 문제점을 놓침

해결 방법

봇 주문 분산 + 매칭 병렬화 - 선택 ✓	기능별 서버를 분리해 MSA 운영	Redis 파이프라인/배치 주기 조절
장점 • 한정된 자원 내 즉각적인 병목 해결 가능 **단점** • 단일 서비스 내 스레드 동기화 및 데이터 처리를 세밀하게 관리 • 확장성 문제와 근본 문제 해결이 불가능	**장점** • 확장성 문제와 근본적인 문제를 둘 다 해결 할 수 있음 **단점** • 프로젝트 최중 제출일 까지 공수 부족 • 아키텍처 전면 수정에 따른 예상치 못한 변수가 발생 가능성 • 한정된 자원 속 리소스 관리 리스크가 생김	**장점** • 아키텍처 구조 변경 없이 코드레벨 전송 주기 수정 • 단순함 **단점** • 봇 주문의 싱글 스레드 CPU 병목이라는 근본 원인을 해결하지 못함 • 대규모 동시 접속 시 여전히 연결 실패율이 발생할 위험이 큼

구현

- **봇 주문 경로 개선:** Kafka를 경유하던 봇 주문을 Redis에 바로 적재하도록 변경하고, Redis Stream 방식으로 전환해 동일 수량의 주문을 시간축으로 분산, 순간 부하를 낮춤
- **매칭 확인 병렬화**: 스케줄링을 통해 아이템별로 순차 확인하던 매칭 로직을 멀티스레드 기반 아이템별 병렬 처리로 전환, Lua 스크립트와 연계해 데이터 적재율 방어

결과 및 회고

- 동일 조건(k6 200 VU Stress Test)에서 재측정한 결과, WebSocket 연결 실패율 44% → 0% 달성 확인
- 그러나 HPA 스케일 아웃으로 레플리카가 늘어나자 단일 인스턴스를 전제로 작성된 스케줄러봇이 Pod 수만큼 중복 실행 MSA 분리는 일정상 보류하고 ShedLock을 적용해 한 시점에 한 Pod에서만 실행되도록 제어
- 스케일 아웃은 서버를 늘리는 것으로 끝나지 않고, 코드가 다중 인스턴스에서도 같은 결과를 내는지까지 확인해야 한다는 점을 느꼈습니다
- 앞으로는 설계 단계에서 다중 인스턴스 실행 시 예상되는 문제를 먼저 점검하고, 스케줄러처럼 한 번만 실행되어야 하는 로직은 분리해 설계하겠습니다

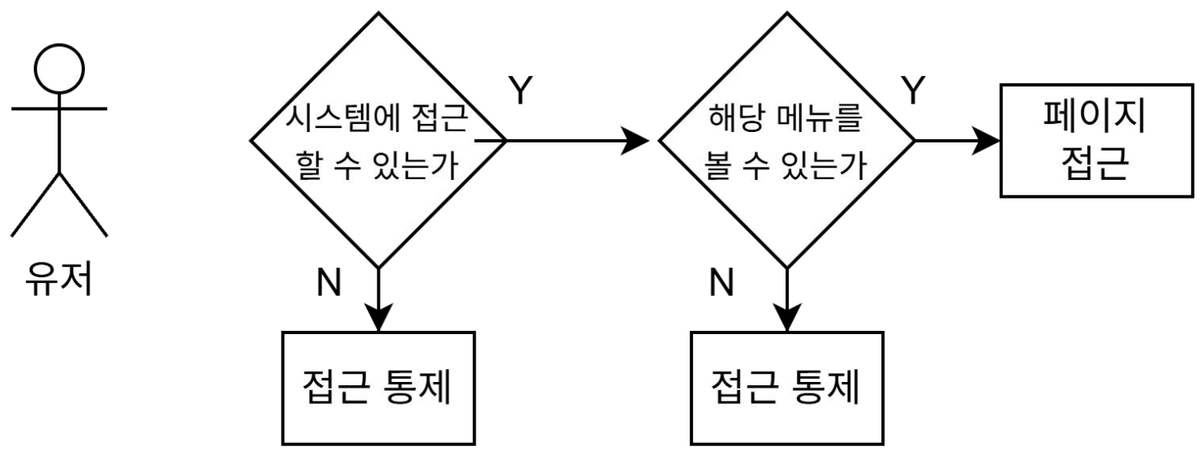
URL 레벨 권한 검증 부재 취약점 해결

모의해킹 진단에서 지적된 인가 취약점을 해결한 경험

2026.08 역할: 개발자 프로젝트: TMS MES

문제 상황

- 모의해킹 진단에서 URL 레벨 권한 검증 부재 취약점 발견
- 메뉴와 권한이 부여되지 않아도 url 직접 호출 시 접근 가능



원인 분석

- 기존 권한 관리가 시스템 접속 시 확인, 유저 별 role에 따른 보여주는 메뉴로 통제 중
- url 단위의 체크는 미존재

AI 한계

- 해당 기능을 구현 시 url_master, role_url 매핑 2개의 테이블로 구현하라고 추천
- 어노테이션을 만들어서 함수 단위의 해결책을 제시

해결 방법

기존 role과 결합 + 인터셉터 배치 - 선택 ✓	2개의 테이블 + 함수별 어노테이션	Spring Security로 URL 인가 적용
장점 • 모든 요청이 한 지점을 지나 누락이 불가능 • URL 목록을 사전이 정의하지 않아도 됨 **단점** • 캐시와 DB간 갱신 지점 불일치 가능성 • 직접 구현으로 유지보수 시 어려울 수 있음	**장점** • 권한이 함수에 명시되어 직관적 • 함수레벨의 하위 접근 통제 가능 **단점** • 어노테이션 누락 가능성 존재 • 수정 범위가 큼 • 권한이 코드와 DB 이중 관리 되어야함	**장점** • 검증된 표준 프레임워크 • 유지보수 및 보안 업데이트를 받을 수 있음 **단점** • 기존 인증 체계 변경 필요 • 모든 url 패턴에 대한 분석 필요 • DB 기반 동적 권한 부여 커스텀 구현 필요

구현

- URL 테이블 신규 구성: URL 테이블 1개로 요청된 URL이 테이블에 없으면 자동 적재, role id를 복수 지정할 수 있도록 구성, 권한 미설정 url은 admin만 접속 가능
- 권한 정보 캐싱: 권한 정보를 캐시로 조회해 매 요청마다 DB를 거치지 않도록 구성, 화면에 캐시 갱신 버튼 구현
- URL별 호출 카운트: 호출마다 +1 하여 URL당 호출 횟수를 파악 (추후 프로그램 개선 시 사용하기 위해 추가) - 해당 부분 추후 속도 저하 발생 시 배치 처리를 통해 개선 예정

결과 및 회고

- 모의해킹 재진단에서 URL별 권한 지정 항목 통과
- 요청 시 자동 적재된 URL 400개 이상 확보
- 신규 URL도 별도 작업 없이 목록에 포함되는 구조 마련
- 권한 검증을 위해 만든 기능에 호출 횟수 수집을 더하니 사용 현황 파악에도 쓸 수 있다는 점을 느껴 앞으로는 기능을 만들 때 요구사항에 더해

  적은 비용으로 얻을 수 있는 부가 가치가 있는지 함께 고민하겠습니다.

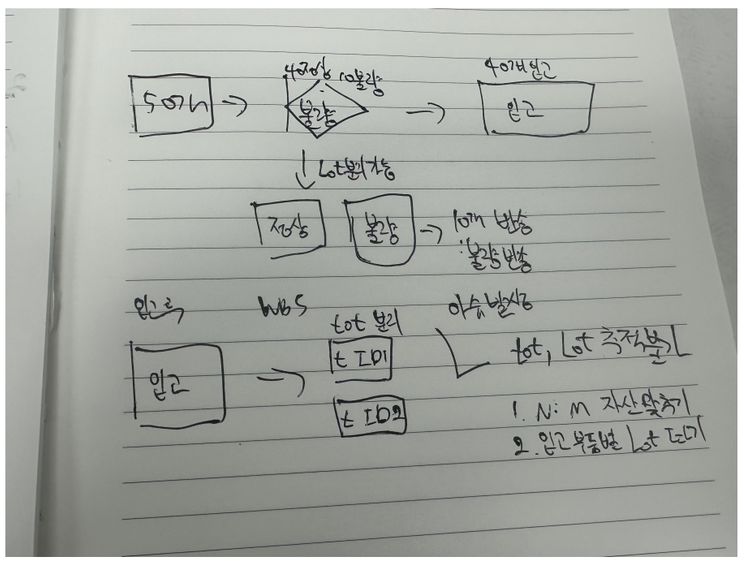
요구사항 분석·구조화

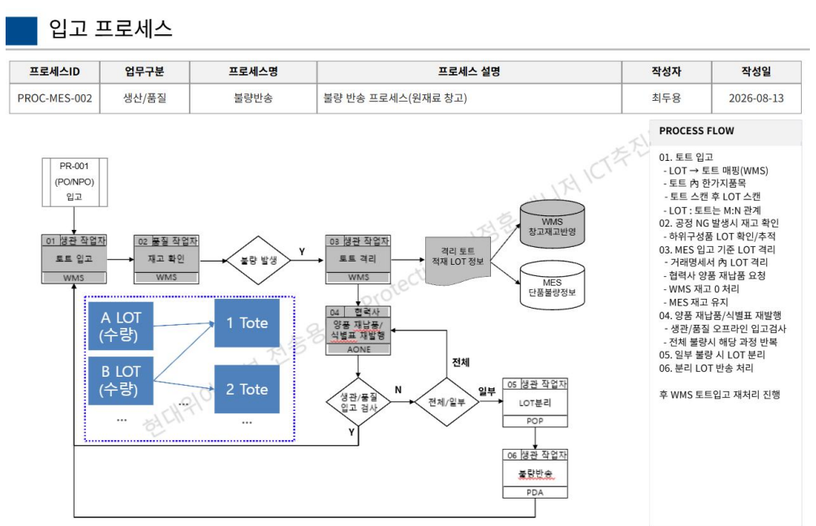
프로세스 흐름도 작성

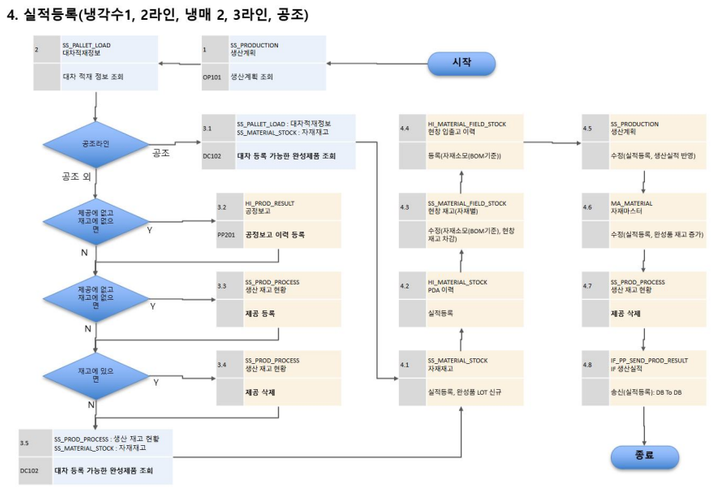
2026.04 — ing　역할: 개발자　프로젝트: TMS / MES 시스템

문제 상황

- 복잡한 기능들의 흐름을 이해하기 위해서 코드만으로 따라가는 상황, 시간이 오래 걸림
- 프로세스를 만들 시 구두로 논의하며 했던 이야기가 반복되거나 예외 상황을 당시 고려 못하고 또 다시 이야기하는 상황 발생







원인 분석

- 프로세스의 문서가 없다 보니 흐름을 이해하기 위해 시간이 오래 걸림, 코드에 대한 이해도도 낮아짐
- 각 담당자들 머리속에 있는 내용을 말하기에 배경 지식들이 달라 이해하는데 어려움을 겪음

AI 한계

- 가독성 부족: AI 에게 생각하는 흐름을 도식화로 요청했지만 한 눈에 알아보기 힘든 구조로 생성
- 도메인·맥락 측면의 한계: 언급되지 않은 예외사항 및 사내 시스템에 대한 맥락 부재, 그럴 듯 하게만 작성

해결 방법

- 담당자들이 같은 화면을 보고 이야기 하기 위해 문서를 작성하고자 함
- 구두로만 오가는 복잡한 업무 프로세스를 놓치지 않기 위해 현장 로직을 직접 노트에 도식화하며 흐름과 예외 케이스를 정리
- 정리한 사항을 Process Flow와 인터페이스 정의서로 작성해 구체화

구현 및 검증

- 작성한 flow를 바탕으로 시니어 개발자와, PM과 리뷰를 진행하며 타당성을 검증함
- 현업 담당자와 함께 검토하며 실제 현장 흐름 및 예외 케이스가 빠짐없이 반영 되었는지 최종 확인
- 최종 확인 된 flow는 DB와 연결하여 고도화해 문서로 남김

결과

- 개발 도중 발생할 수 있었던 요구사항 누락 및 예외 상황 위험 차단
- flow를 보면서 소통해 커뮤니케이션 오류 해결
- AI 활용 개발 생산성 향상, 시스템 확장성 및 유지보수성 확보
- 본격적인 개발 전 프로세스 도식화 습관 마련